plotly

python -m venv venv

.\venv\Scripts\Activate.ps1

pip install dash plotly pandas

pip freeze > requirements.txt

realtime-iot-dashboard/ %

% % % venv/ %

% % % app.py %

% % % data_generator.py %

% % % requirements.txt

% % % % assets/

% % % style.css

import random

from datetime import datetime

def generate_sensor_data():

temperature = round(random.uniform(22, 35), 2)

humidity = round(random.uniform(40, 80), 2)

pressure = round(random.uniform(990, 1030), 2)

return {

"time": datetime.now().strftime("%H:%M:%S"),

"temperature": temperature,

"humidity": humidity,

"pressure": pressure

}

app.py

from dash import Dash, html, dcc, Input, Output Dash Core Components.

import plotly.graph_objects as go

from collections import deque

from data_generator import generate_sensor_data

--

Initialize Dash

--

```
app = Dash(__name__)
app.title = "Real-Time IoT Dashboard"
# ------------------------------------------------
# Store recent sensor readings
# ------------------------------------------------
MAX_POINTS = 30
times = deque(maxlen=MAX_POINTS)
temperatures = deque(maxlen=MAX_POINTS)
humidities = deque(maxlen=MAX_POINTS)
pressures = deque(maxlen=MAX_POINTS)
# ------------------------------------------------
# Dashboard Layout
# ------------------------------------------------
app.layout = html.Div(
className="dashboard",
children=[
# Header
html.Div(
className="header",
children=[
html.H1("IoT Real-Time Monitoring"),
html.P("Live Sensor Analytics Dashboard")
]
),
# KPI Cards
html.Div(
className="cards",
children=[
html.Div(
className="card",
children=[
html.H4("Temperature"),
html.H2(id="temperature-value"),
```

```
html.P("°C")
]
),
html.Div(
className="card",
children=[
html.H4("Humidity"),
html.H2(id="humidity-value"),
html.P("%")
]
),
html.Div(
className="card",
children=[
html.H4("Pressure"),
html.H2(id="pressure-value"),
html.P("hPa")
]
),
html.Div(
className="card",
children=[
html.H4("System Status"),
html.H2(
```

"%Ï ONLINE",

```
className="online"
),
html.P("Real-time monitoring")
]
)
]
),
# Temperature chart
```

```
html.Div(
className="chart-container",
children=[
dcc.Graph(id="temperature-chart")
]
),
# Second row
html.Div(
className="chart-row",
children=[
html.Div(
className="chart-half",
children=[
dcc.Graph(id="humidity-chart")
]
),
html.Div(
className="chart-half",
children=[
dcc.Graph(id="pressure-chart")
]
)
]
),
# Update indicator
html.Div(
className="footer",
children=[
html.Span("%Ï LIVE"),
html.Span(id="last-update")
]
),
# Timer
```

```
dcc.Interval(
id="interval-component",
interval=2000,
n_intervals=0
)
]
)
# ------------------------------------------------
# Real-time callback
# ------------------------------------------------
@app.callback(
[
Output("temperature-value", "children"),
Output("humidity-value", "children"),
Output("pressure-value", "children"),
Output("temperature-chart", "figure"),
Output("humidity-chart", "figure"),
Output("pressure-chart", "figure"),
Output("last-update", "children")
],
Input("interval-component", "n_intervals")
)
def update_dashboard(n):
# Generate new sensor reading
sensor = generate_sensor_data()
# Store data
times.append(sensor["time"])
temperatures.append(sensor["temperature"])
humidities.append(sensor["humidity"])
pressures.append(sensor["pressure"])
# ------------------------------------------------
# Temperature graph
# ------------------------------------------------
```

```
temperature_fig = go.Figure()
temperature_fig.add_trace(
go.Scatter(
x=list(times),
y=list(temperatures),
mode="lines+markers",
name="Temperature"
)
)
temperature_fig.update_layout(
title="Live Temperature",
xaxis_title="Time",
yaxis_title="Temperature (°C)",
template="plotly_dark"
)
# ---------------------------------------------
# Humidity graph
# ---------------------------------------------
humidity_fig = go.Figure()
humidity_fig.add_trace(
go.Scatter(
x=list(times),
y=list(humidities),
mode="lines+markers",
name="Humidity"
)
)
humidity_fig.update_layout(
title="Live Humidity",
xaxis_title="Time",
yaxis_title="Humidity (%)",
template="plotly_dark"
)
```

```
# ---------------------------------------------
# Pressure graph
# ---------------------------------------------
pressure_fig = go.Figure()
pressure_fig.add_trace(
go.Scatter(
x=list(times),
y=list(pressures),
mode="lines+markers",
name="Pressure"
)
)
pressure_fig.update_layout(
title="Live Pressure",
xaxis_title="Time",
yaxis_title="Pressure (hPa)",
template="plotly_dark"
)
return (
f"{sensor['temperature']:.2f}",
f"{sensor['humidity']:.2f}",
f"{sensor['pressure']:.2f}",
temperature_fig,
humidity_fig,
pressure_fig,
f"Last updated: {sensor['time']}"
)
# ---------------------------------------------
# Run application
# ---------------------------------------------
if __name__ == "__main__":
app.run(debug=True)
```

Style.css

```
body {
margin: 0;
font-family: Arial, sans-serif;
background-color: #111827;
color: white;
}
.dashboard {
padding: 25px;
}
.header {
text-align: center;
margin-bottom: 30px;
}
.header h1 {
font-size: 32px;
margin-bottom: 5px;
}
.header p {
color: #9ca3af;
}
.cards {
display: grid;
grid-template-columns: repeat(4, 1fr);
gap: 20px;
margin-bottom: 25px;
}
.card {
background-color: #1f2937;
padding: 20px;
border-radius: 12px;
text-align: center;
box-shadow: 0 4px 15px rgba(0, 0, 0, 0.3);
}
```

```
.card h4 {
color: #9ca3af;
margin-bottom: 10px;
}
.card h2 {
font-size: 30px;
margin: 5px;
}
.card p {
color: #9ca3af;
}
.online {
color: #22c55e;
}
.chart-container {
background-color: #1f2937;
border-radius: 12px;
padding: 10px;
margin-bottom: 20px;
}
.chart-row {
display: grid;
grid-template-columns: 1fr 1fr;
gap: 20px;
}
.chart-half {
background-color: #1f2937;
border-radius: 12px;
padding: 10px;
}
.footer {
display: flex;
justify-content: space-between;
```

```
margin-top: 20px;
color: #9ca3af;
}
.footer span:first-child {
color: #22c55e;
}
```